AIVision 03강 복습 질문: 정답 및 해설

1. 데이터 탐색

Q1. 모델을 학습하기 전에 데이터를 먼저 탐색하는 이유는 무엇일까요?

일반적인 목적

데이터 탐색은 데이터의 구조, 값의 특성, 결측값, 클래스 분포 등을 파악하여 적절한 전처리와 학습 방법을 결정하기 위한 과정입니다.

이번 실습에서의 목적

이번 실습에서는 전체 데이터 중 **어떤 데이터를 선택하여 학습용 서브셋(Subset)을 구성할 수 있는지 파악하는 것**이 주요 목적이었습니다.

웨이퍼맵의 크기별 분포, 결함 유형별 개수 등을 확인하여 다음 사항을 결정할 수 있었습니다.

- 어떤 이미지 크기의 데이터를 사용할 것인가?
- 어떤 결함 클래스를 학습할 것인가?
- 각 클래스에서 얼마나 많은 데이터를 확보할 수 있는가?

즉, 데이터를 탐색한 결과를 바탕으로 **학습에 사용할 데이터를 선택하는 기준**을 정했습니다.

Q2. pickle 파일을 불러온 뒤 `df.shape`, `df.columns`, `df.head()`, `df.info()`를 실행한 이유는 무엇일까요?

코드	확인할 수 있는 정보
`df.shape`	데이터의 행과 열 개수
`df.columns`	컬럼 이름
`df.head()`	처음 몇 행의 실제 데이터
`df.info()`	컬럼별 데이터 타입, 결측이 아닌 값의 개수 등

이러한 정보를 확인하면 데이터가 어떻게 구성되어 있는지 파악하고, 이후 분석에 필요한 컬럼을 결정할 수 있습니다.

참고로 `df.info()`에서 `waferMap`의 타입이 `object`로 표시되더라도, 각 셀에는 실제로 NumPy 배열이 저장되어 있을 수 있습니다.

Q3. `waferMap` 컬럼에는 무엇이 들어 있었나요?

각 행의 `waferMap`에는 웨이퍼의 검사 결과를 나타내는 **2차원 NumPy 배열**이 들어 있습니다.

배열의 각 원소는 웨이퍼의 해당 위치에 대한 값이며, 이번 데이터에서는 다음과 같이 구분됩니다.

- `0`: 웨이퍼 영역 밖
- `1`: 정상 영역
- `2`: 불량 영역

따라서 배열을 이미지로 표현하면 불량이 발생한 위치와 분포를 시각적으로 확인할 수 있습니다.

Q4. `df['waferMap'].apply(lambda x: x.shape)`는 무엇을 확인하기 위한 코드인가요?

각 행에 저장된 웨이퍼맵 배열의 **크기(shape)**를 확인하기 위한 코드입니다.

- `apply()`는 각 행의 값에 함수를 적용합니다.
- `lambda x: x.shape`는 해당 배열의 크기를 반환합니다.

웨이퍼맵마다 크기가 다를 수 있으므로, 이를 확인해야 이미지 크기별 데이터 분포를 분석하거나 특정 크기의 데이터만 선택할 수 있습니다.

---

2. 데이터 분포 이해

Q5. `failureType`의 개수를 확인한 이유는 무엇일까요?

각 결함 유형에 해당하는 데이터가 얼마나 있는지 확인하기 위해서입니다.

이번 실습에서는 이 정보를 통해 다음을 판단할 수 있었습니다.

- 어떤 결함 유형의 데이터가 충분히 있는가?
- 학습에 사용할 클래스를 어떻게 선택할 것인가?
- 클래스별 데이터 개수를 비슷하게 구성할 수 있는가?

즉, 클래스별 개수 확인도 **학습용 서브셋을 설계하기 위한 탐색 과정**이었습니다.

Q6. 클래스별 데이터 개수가 크게 다르면 CNN 학습 결과에 어떤 문제가 생길 수 있을까요?

학습 데이터가 많은 클래스의 특징을 모델이 더 많이 학습하게 되어, 해당 클래스로 예측이 치우칠 수 있습니다.

이 경우 전체 정확도는 높더라도 데이터가 적은 클래스의 분류 성능은 낮을 수 있습니다.

따라서 클래스별 데이터 분포를 확인하고, 필요하다면 클래스별 샘플 수를 조정하는 등의 방법을 고려해야 합니다.

---

3. 전처리

Q7. `failureType_clean` 컬럼을 새로 만든 이유는 무엇일까요?

원래 `failureType`에는 결함 유형이 배열 형태로 저장되어 있어, 그대로 분석에 사용하기 불편했습니다.

이를 `Center`, `Edge-Ring`, `Loc` 등의 **문자열 형태로 정리한 컬럼**이 `failureType_clean`입니다.

이렇게 하면 클래스별 개수를 확인하거나, 특정 클래스를 선택하거나, 그룹별로 데이터를 분석하기 쉬워집니다.

Q8. `waferShape` 컬럼을 새로 만든 이유는 무엇일까요?

각 행에 들어 있는 `waferMap` 배열의 크기를 별도 컬럼으로 저장하여 **크기별 분포를 분석하고 데이터를 필터링하기 쉽게 하기 위해서**입니다.

```
df['waferShape'] = df['waferMap'].apply(lambda x: x.shape)
```

이후에는 다음과 같이 원하는 크기의 데이터를 간단하게 선택할 수 있습니다.

```
df[df['waferShape'] == (26, 26)]
```

`waferShape` 컬럼이 없더라도 배열의 크기를 직접 확인하여 필터링할 수 있지만, 매번 계산해야 하므로 불편합니다.

Q9. `waferMap`과 `waferShape` 컬럼의 차이는 무엇인가요?

컬럼	저장된 정보	예시
`waferMap`	웨이퍼의 검사 결과를 나타내는 2차원 배열	`[[0, 1, 1], [0, 2, 1], ...]`
`waferShape`	해당 배열의 행과 열 개수	`(26, 26)`

`waferMap`은 **이미지로 표현할 실제 데이터**이고, `waferShape`는 **그 데이터의 크기를 설명하는 정보**입니다.

---

4. 데이터 필터링

Q10. 지난 실습에서는 왜 `(26, 26)` 크기의 웨이퍼맵만 선택했을까요?

CNN은 한 번에 학습하는 이미지들이 동일한 입력 크기를 갖도록 구성해야 합니다.

지난 실습에서는 크기가 같은 `(26, 26)` 웨이퍼맵을 선택하여 **별도의 크기 변환 없이 학습 데이터를 구성**했습니다.

이것은 CNN에 사용할 수 있는 데이터가 `(26, 26)`뿐이라는 뜻은 아닙니다. 실습을 단순하게 진행하기 위한 데이터 선택 방법이었습니다.

Q11. `(26, 26)`이 아닌 웨이퍼맵은 학습에 사용할 수 없을까요?

사용할 수 있습니다.

웨이퍼맵의 원래 크기가 다르더라도 CNN에 입력하기 전에 동일한 크기로 변환하면 됩니다.

예를 들어 다양한 크기의 웨이퍼맵을 모두 `(64, 64)`로 변환할 수 있습니다.

이때 크기 변환 과정에서 결함의 형태나 위치 정보가 어떻게 달라지는지도 고려해야 합니다.

오늘 실습에서는 서로 다른 크기의 웨이퍼맵을 이미지로 변환하고, 크기를 통일하여 학습하는 방법을 다룹니다.

---

5. 전처리 결과 저장

Q12. 전처리한 데이터를 다시 pickle 파일로 저장한 이유는 무엇일까요?

전처리 결과를 저장해두면 다음 실습에서 같은 작업을 다시 실행하지 않고, **저장된 데이터를 바로 불러와 사용할 수 있습니다.**

특히 데이터 탐색과 전처리에 시간이 오래 걸리는 경우 중간 결과를 저장해두면 실습을 효율적으로 진행할 수 있습니다.

또한 pickle 형식은 DataFrame과 그 안에 포함된 배열 등의 파이썬 객체를 저장하는 데 사용할 수 있습니다.

Q13. 원본 pickle 파일을 덮어쓰지 않고 새로운 파일로 저장하는 것이 좋은 이유는 무엇일까요?

원본 데이터를 보존하기 위해서입니다.

전처리 과정에서 데이터를 삭제하거나 변형했더라도 원본이 남아 있으면 다른 기준으로 다시 처리할 수 있습니다.

예를 들어 이번에는 `(26, 26)` 데이터만 선택했지만, 이후 다른 크기의 웨이퍼맵을 사용하려면 원본 데이터가 필요합니다.

따라서 일반적으로 원본 데이터와 전처리 결과는 구분해서 보관하는 것이 좋습니다.

---

6. 종합 질문

Q14. 지난 시간의 처리 과정 중 하나를 제거해야 한다면, 어떤 단계가 가장 문제가 될 것 같나요?

정해진 정답은 없습니다.

어떤 단계를 선택했는지보다 **그 단계의 역할과 제거했을 때의 영향을 설명할 수 있는지**가 중요합니다.

답변 예시:

- **데이터 탐색**: 데이터의 특성을 모르기 때문에 적절한 서브셋을 구성하기 어렵습니다.
- `failureType_clean` **생성**: 결함 유형별 분석과 클래스 선택이 불편해집니다.
- `waferShape` **생성**: 크기별 분포를 확인하고 특정 크기의 데이터를 선택하는 작업이 번거로워집니다.
- `(26, 26)` **필터링**: 서로 다른 크기의 데이터를 함께 사용하려면 별도의 크기 변환 작업이 필요합니다.
- **전처리 결과 저장**: 다음 실습에서 전처리 과정을 다시 실행해야 할 수 있습니다.

중요한 것은 각 단계가 독립된 작업이 아니라 **데이터 탐색 → 전처리 → 서브셋 구성 → 저장**이라는 하나의 흐름으로 연결된다는 점입니다.